

Lecture 1

Inside a Modern LLM: From Byte to Attention

Tokenization, embeddings, positions, and the two costs of attention

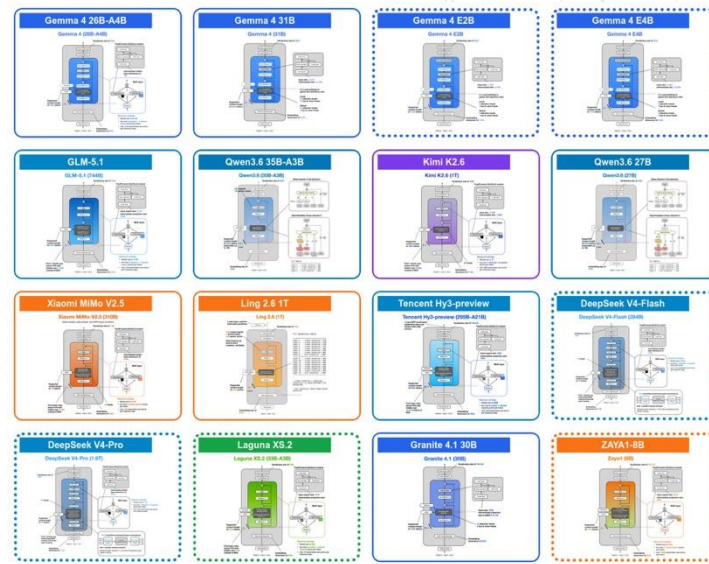
Gleb Molodtsov · 7 September 2026



Three flagship releases recently— three different attentions

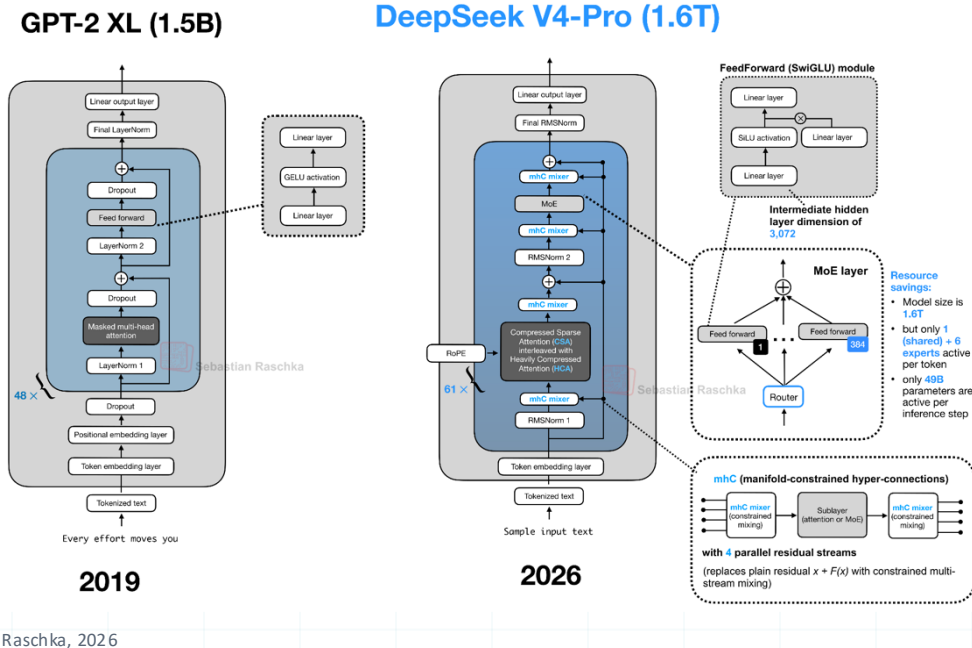
Three months before this lecture, three open releases, three incompatible answers to one question: how not to pay for long context.

- **DeepSeek V4** (Apr) — compresses the KV cache along the sequence: CSA + HCA
- **Kimi K3** (Jul) — 2.8T parameters on linear KDA + attention residuals
- **Qwen3.8-Next** (Aug) — 3:1 Gated DeltaNet and Gated Attention, dense 28B
- None of the three runs the attention from the 2017 paper.



Raschka, 2026, «Recent Developments in LLM Architectures»

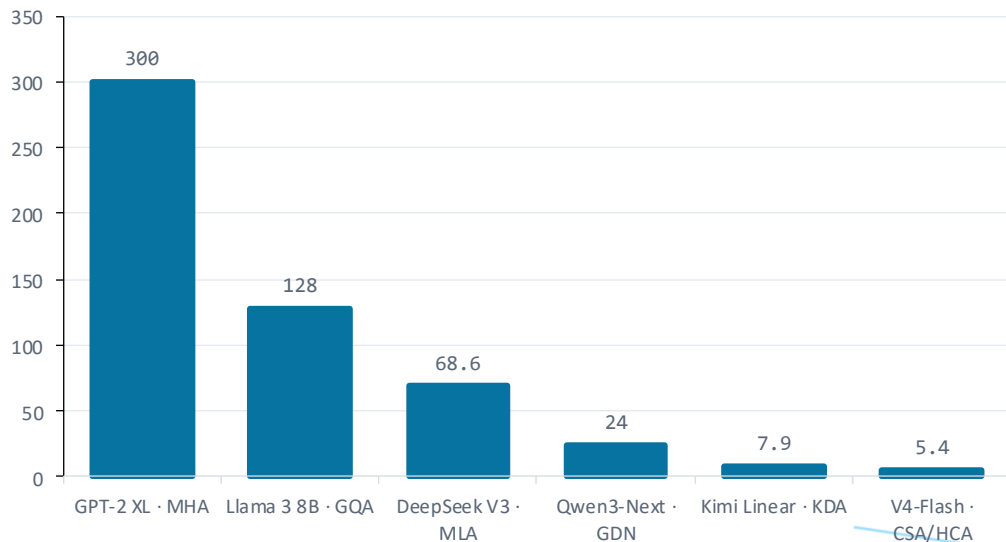
Seven years changed everything except the skeleton



A stack of identical blocks, an additive residual stream, a next-token loss — that did not change. Everything inside the block did.



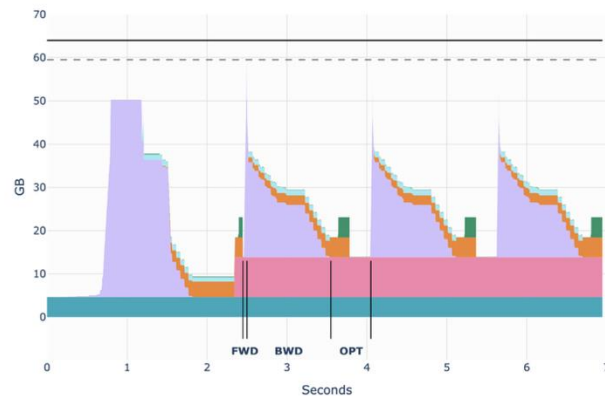
KV cache per token fell 55× in seven years



Computed by Raschka, LLM Architecture Gallery — 97 models, updated 23 Aug 2026

READ IT AS ONE SENTENCE

- The model grew from 1.5B to 284B parameters
- The context grew from 1k to 1M tokens
- The cache per token shrank by a factor of 55



HF UltraScale Playbook





Architecture changes of the last five years are payments on three axes



$O(n^2)$ compute at training · KV-cache memory at inference · bandwidth of the residual stream



By the end you read a frontier config without a glossary

YOU WILL KNOW THESE

- BPE, fertility, vocabulary size, weight tying
- RoPE, the θ base, YaRN, NoPE
- MHA $\rightarrow$ MQA $\rightarrow$ GQA $\rightarrow$ MLA
- KV cache, and how to size it in GB
- Sliding window, DSA, CSA / HCA

THE NEXT LECTURE

- FFN, SwiGLU, MoE and shared experts
- Normalisation: Pre-RMSNorm, QK-Norm
- Hyper-connections and mHC
- Sampling beyond one slide
- The full parameter budget

```
{ "model_type": "deepseek_v4",      "moe_intermediate_size": 2048,      "index_n_heads": 64,      "max_position_embeddings": 1048576,      "quantization_config": {
  "hidden_size": 4096,              "num_attention_heads": 64,          "index_head_dim": 128,      "compress_rope_theta": 160000,      "quant_method": "fp8",
  "num_hidden_layers": 43,          "num_key_value_heads": 1,          "index_topk": 512,         "rope_scaling": {              "fmt": "e4m3",
  "n_routed_experts": 256,          "head_dim": 512,                  "expert_dtype": "fp4",      "factor": 16,                  "weight_block_size": [128, 128]
  "n_shared_experts": 1,           "q_lora_rank": 1024,              "hc_mult": 4,              "type": "yarn"                }
  "num_experts_per_tok": 6,         "o_lora_rank": 1024,              "hc_sinkhorn_iters": 20,   },                                }
  "qk_rope_head_dim": 64,
```

huggingface.co/deepseek-ai/DeepSeek-V4-Flash/blob/main/config.json



Fifteen lectures follow the life of one model

1-2 Inside a modern LLM

3 Pre-training I: data, scaling laws

4 Pre-training II: efficient training

5 SFT, alignment, in-context learning

6 Basic RL: MDP, policy gradient, PPO

7 Advanced RL and reasoning: GRPO, RLVR

8 Evaluation: measuring the unmeasurable

9 RAG and production

10 Agents

11 General efficiency: quantization, distillation

12 Inference efficiency: KV, speculative decoding

13 Safety, interpretability, security

14 Diffusion LM

15 VLM and multimodality



One model runs through the entire semester

01

Building

Block architecture, attention, parameter budget.

Lectures 1-2 • HW 1

02

Training

Data, scaling laws, optimizers, parallelism.

Lectures 3-4 • HW 2

03

Alignment & Evaluation

SFT, PEFT, RLHF, GRPO, reasoning, evaluation.

Lectures 5-8 • HW 3-4

04

Production

RAG, agents, inference efficiency.

Lectures 9-12 • HW 5-6

05

Interpretability & Applications

VLM, Safety, diffusion LM



FORMAT

Mondays: lecture at 13:55, seminar at 15:30

LECTURE · 13:55

- 90 minutes, blocks of 15, a break in the middle
- Slides go to the course repository the same day
- Questions out loud, mid-slide, not at the end

SEMINAR · 15:30

- A notebook with a skeleton and unit tests — we write code
- Seminar 1: research tooling; then one per lecture topic
- Seminar 2: a transformer from scratch — attention, mask, KV cache



ADDITIONAL-CHAPTERS-NLP-COURSE Public



main ▾



1 Branch



0 Tags

<https://github.com/BRAIn-Lab-teaching/ADDITIONAL-CHAPTERS-NLP-COURSE>



12 points from 3 sources

6

homework

Six tasks with an oral defence. The bulk of the grade.

5

project

Research or applied, defended at the end of the semester.

1

reading

One paper presentation at the reading group.



The soft deadline is two weeks; there are only two hard ones

- Each homework is issued for **two weeks** — that is the soft deadline
- Missing the soft one costs nothing: the work is still accepted
- **26 October, 23:59** — hard deadline for the first three tasks
- **7 December, 23:59** — hard deadline for the second three
- After a hard deadline that batch of three is closed for good



**You draw one homework out of three — all
three are graded on that answer**

you defend it — all three counted · you don't — none of the three counted



Project

TWO FORMATS

- **Research** — aimed at a paper: hypothesis, experiment, analysis
- **Applied** — a working system where an LLM solves a real task
- Topic agreed in person, in the first 2 weeks

EXAM

- An oral exam over the whole syllabus instead of the project
- Format: a conversation over the programme
- Most of you have used LLMs as a tool. Research with LLMs is a different skill — the project exists for exactly that.



The reading group presentation implies an in-depth analysis of the field, not a retold abstract

30 minutes:

- **The field.** Where the area stands, which works the paper builds on, whom it competes with.
- **The problem.** The exact task, why it's hard, why prior work falls short.
- **The method.** Every key component — what it does, why it's there, what breaks without it.
- **The experiments.** Setups, baselines, metrics, ablations — and an active hunt for weak spots: stale or unfair baselines, missing ablations, narrow task/scale coverage, cherry-picking, text-vs-table mismatches.
- **Tech reports.** All key innovations (architecture, data, training stages, infra), plus where the model actually sits among the others — where it wins, where it loses, where the comparison is rigged.

Reproducing an experiment or applying the method is highly encouraged.

DeepSeek-V4:

Towards Highly Efficient Million-Token Context Intelligence

DeepSeek-AI
research@deepseek.com

REINFORCEMENT LEARNING WITH VERIFIABLE REWARDS IMPLICITLY INCENTIVIZES CORRECT REASONING IN BASE LLMs

Xumeng Wen^{*1}, Zihan Liu^{*†2}, Shun Zheng^{*†1}, Shengyu Ye^{†1}, Zhirong Wu¹, Yang Wang¹, Zhijian Xu^{†3}, Xiao Liang^{†4}, Junjie Li¹, Ziming Miao¹, Jiang Bian¹, Mao Yang¹

¹Microsoft Research Asia ²Peking University

³The Chinese University of Hong Kong ⁴University of California, Los Angeles

Dion3: Full-stack orthogonal updates

Noah Amsel^{*1}, Jack Zhang², Kwangjun Ahn^{*3}, Ali Naeimi⁴, Austin Feng^{*5}, Berlin Chen², Tri Dao², and John Langford⁶

Bridging the Black Box: A Survey on Mechanistic Interpretability in AI

01

- **The task: predict the next token**

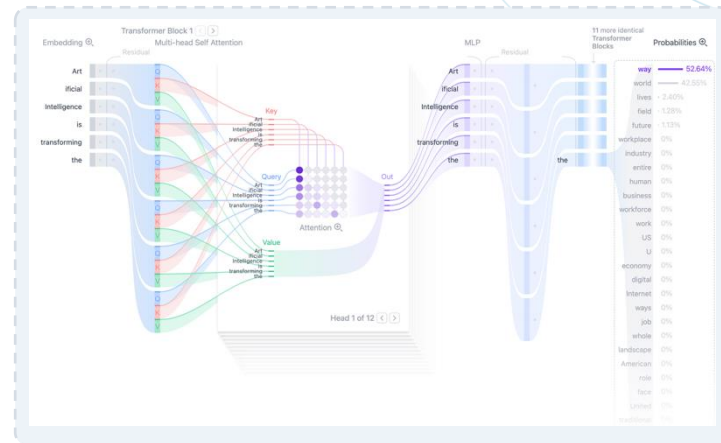
What exactly is optimised, and how it is measured — before we discuss what computes it



An LLM is one distribution over the next token

The model maps a prefix to a vector of V numbers. Everything else is how we get that vector and what we do with it.

- $p(x_1 \dots x_n) = \prod p(x_t | x_{<t})$ — the chain rule, no approximation
- The loss is cross-entropy over all positions at once
- Chat, agents, reasoning — all of it is the same sample-the-next-token loop



Transformer Explainer, GPT-2 small · poloclub.github.io/transformer-explainer

Decoder-only won on interface and inference, not on parallelism

WHAT THE ALTERNATIVES ACTUALLY COST

- **Masked LM** — the loss covers ~15% of positions per forward: far fewer targets per FLOP
[MASK] is seen in training and never at inference — a train/test mismatch
- **Encoder-decoder** — parameters split across two stacks, and every task needs an input/output boundary
Neither loses on parallelism: both train fully parallel over positions

WHAT ACTUALLY DECIDED IT

- One interface: every task is a string continuation — no task head, no per-task finetuning
- Causality makes the prefix reusable: token t never depends on $t+1$, so a KV cache exists
- At matched scale, best zero-shot after pretraining alone; encoder-decoder lagged far behind
- Fewer knobs that interact with scale: no mask ratio, no span length, no depth split



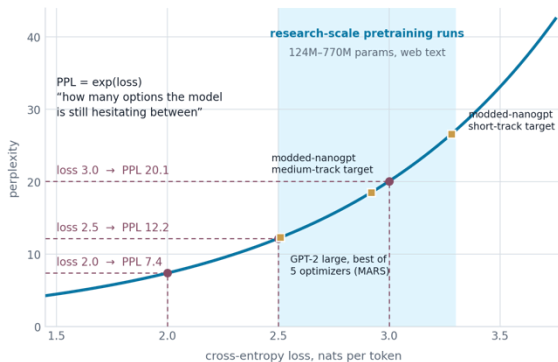
Perplexity is the effective number of choices per step

THE INTUITION

- $PPL = \exp(\text{mean cross-entropy per token})$
- $PPL = 1$: confident and right. $PPL = V$: guessing uniformly
- $PPL = 12$ reads as «the model hesitates between ~12 options»
- Loss 2.5 nats $\approx$ PPL 12.2 — keep the conversion handy

WHERE IT STOPS WORKING

- Perplexity says nothing about factuality or instruction following
- It is not comparable across tokenizers — next slide
- Lecture 8 is entirely about what benchmarks do and do not measure



Curve is exact. Anchors: modded-nanopt speedrun targets on FineWeb (3.28 / 2.92); GPT-2 large on OpenWebText, 2.511-2.606 across optimizers



Same text, two tokenizers. Whose perplexity is lower?

The one with the smaller vocabulary

The one with the larger vocabulary

Equal: perplexity is already normalised

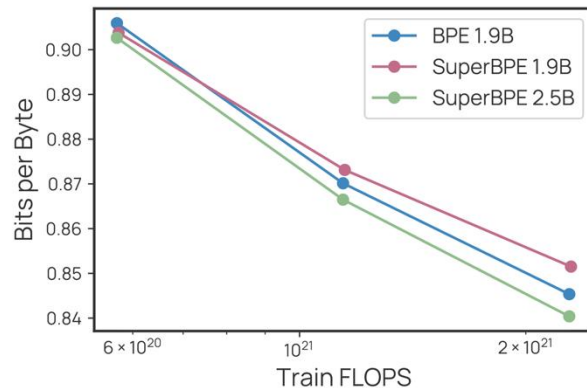
Both options are possible

Answer Both options are possible

PPL is measured per token. A larger vocabulary typically tokenizes the same text into fewer tokens, but each token represents more information and can therefore be harder to predict. Consequently, PPL values obtained with different tokenizers are not directly comparable.

USE THIS INSTEAD

- **Bits-per-byte** — the loss normalised by bytes of text, not by tokens
- BPB compares across any tokenizers, including byte-level models
- SuperBPE and the byte-level literature are built on exactly this metric



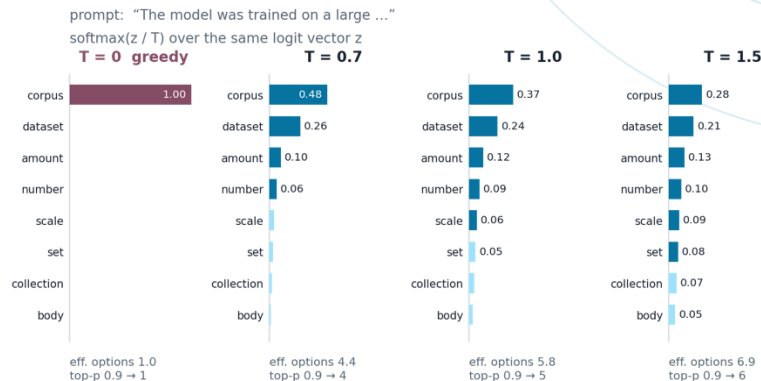
SuperBPE, Fig. 5



One distribution produces very different texts

Sampling does not change the model — it changes which part of it you see.

- **greedy** — argmax, reproducible, prone to loops
- **temperature** — logits divided by T ; $T \rightarrow 0$ is greedy, $T > 1$ flattens
- **top-k / top-p** — cut the tail, then renormalise
- Half the «quality regressions» in practice are someone else's sampling defaults



02

• Before the transformer block: tokenization

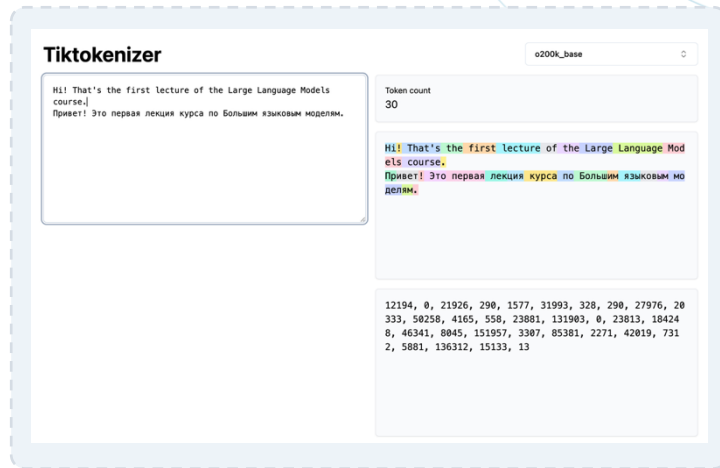
The most underrated part of the stack — and the only one that moves your product's bill by a factor of 1.7



The model never sees letters — it sees integers

A tokenizer is not preprocessing. It is the model's interface to language, fixed before training and kept for the model's whole life.

- text → bytes → token ids → rows of the embedding matrix
- The tokenizer is trained separately, before the model, on its own corpus
- Anything outside the vocabulary is shattered — and the model pays in length



Tiktokenizer, o200k_base

BPE is four lines of a greedy algorithm

```
# training the vocabulary
tokens = list(text.encode("utf-8"))
for i in range(n_merges):
    pair = most_frequent_pair(tokens)
    tokens = merge(tokens, pair, 256 + i)
    merges[pair] = 256 + i
```

minbpe, Karpathy · github.com/karpathy/minbpe

WHAT ACTUALLY MATTERS HERE

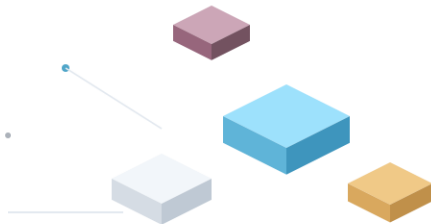
- No linguistics: only pair frequencies in a corpus
- The order of merges is fixed — that order is the vocabulary
- The tokenizer corpus $\neq$ the model corpus, and that is a source of bugs
- Encoding text = replay the same merges in the same order

The same two words, after 0 / 100 / 1 000 / 10 000 merges

byte-level BPE trained here on 1.3 MB of English (Wikipedia + Shakespeare)

	tokenization	songwriting
0 merges	tokenization 12	songwriting 11
vocab 256 1.0 bytes / token		
100 merges	tokenization 9	songwriting 7
vocab 356 1.59 bytes / token		
1 000 merges	tokenization 5	songwriting 5
vocab 1 256 2.54 bytes / token		
10 000 merges	tokenization 3	songwriting 2
vocab 10 256 3.64 bytes / token		

Boundaries follow corpus frequency, not morphemes: "to | ken | ization", not "token | ization".
 "songwriting" collapses to two tokens because both halves are frequent — BPE never learned what a word is.



Byte-level BPE won: OOV became impossible, vocabularies grew fourfold



Model	Year	Vocabulary	Why it is interesting
GPT-2	2019	50 257	first popular byte-level BPE
Llama 3	2024	128 256	tiktoken-compatible, a 2.5× jump
Qwen3	2025	151 936	strong multilingual coverage
GPT-4o	2024	200 000	o200k_base, far better on non-Latin
Gemma 3 / 4	2025–26	262 144	largest among open models
ZAYA1-8B	2026	262 272	a huge vocabulary on an 8B model

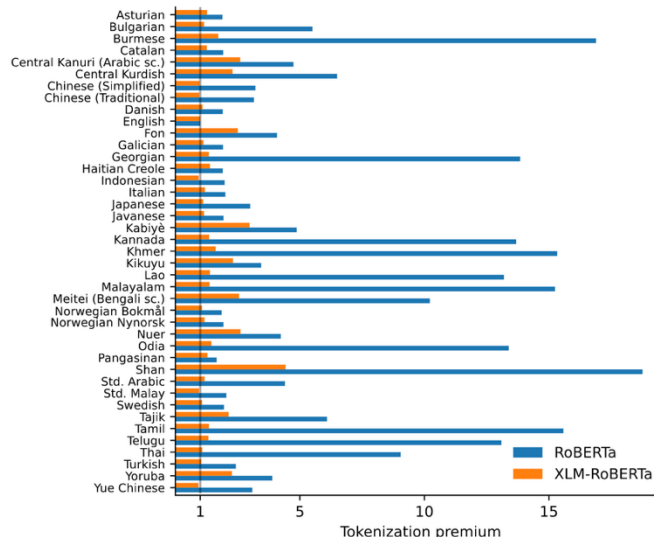
Model configs, collected in the LLM Architecture Gallery (Raschka, updated 23 Aug 2026). WordPiece and Unigram/SentencePiece stayed in the BERT era — know them as context.



The same meaning costs about 70% more in Russian

Not because Russian is «harder» — because the vocabulary was trained on a corpus where English dominated.

- Per 1M words, o200k: English \$2.90, Russian \$4.90 — **+69%**
- French +31%, Hindi +63%, Arabic +70%
- For rare scripts the gap reaches **12–15×**
- Two causes: frequencies in the tokenizer corpus, and UTF-8 bytes per character

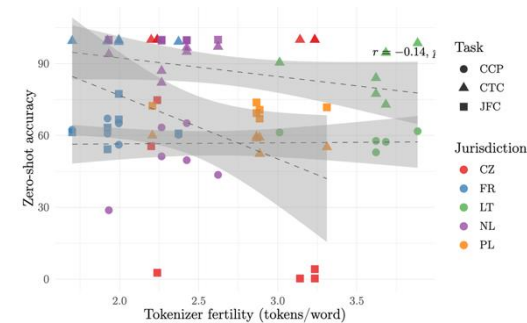


Petrov et al., NeurIPS 2023 — redrawn

Russian does generate slower — but not for the reason people assume

WHAT DOES NOT HAPPEN

- A single decode step does not get slower because of the language
- The model does not «think worse» in Russian because of the tokenizer



Multi-Legal-Bench, 2026, Fig. 5

WHAT ACTUALLY HAPPENS

- The same meaning is **more tokens**, so more decode steps
- Latency $\approx$ token count $\times$ step time $\rightarrow$ higher wall-clock and higher bill
- A 128k context holds **less Russian text** than English
- Longer sequences make attention more expensive — and it grows quadratically

Fertility is a good cost calculator and a poor quality predictor: $r = -0.27$, $p = 0.14$ over 71 model–language pairs (Multi-Legal-Bench, 2026, §6.7)



How many tokens are there in «несовершеннолетний»?

One — it is a frequent word

Two or three — by morphemes

It depends on the tokenizer, and the spread is severalfold

Eighteen — one per letter

Answer It depends on the tokenizer, and the spread is severalfold

We check it live on Tiktokenizer: o200k vs Llama 3 vs Gemma. You also see that none of the three puts boundaries where the morphemes are.

LOOK AT THESE TOGETHER WITH IT

- «strawberry» and letter counting is a tokenizer failure, not a reasoning failure
- Digits are split per character in modern vocabularies — deliberately, for arithmetic
- Glitch tokens: strings in the vocabulary that the model almost never saw in training

Token count
5

Token count
7

Token count
8

несовершеннолетний

несовершеннолетний

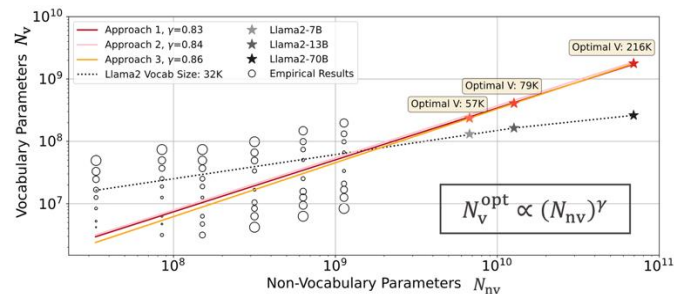
несовершеннолетний

our screenshot, o200k / Llama 3 / Qwen 2.5



Vocabulary size is pulled by three forces at once

- **Larger vocabulary** → shorter sequences → fewer steps, cheaper attention
- **Larger vocabulary** → fatter embedding and LM head: both are $V \times d$
- **Larger vocabulary** → more rare tokens that barely saw a gradient
- For byte-level models the softmax starts to dominate FLOPs between 200k and 400k
- The optimum scales with the model: bigger models deserve bigger vocabularies



Tao et al., 2024, «Scaling Laws with Vocabulary», Fig. 1

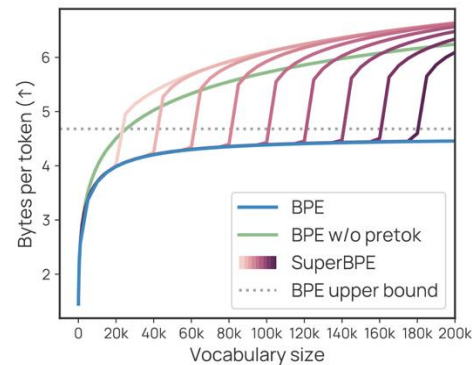
SuperBPE lifted the whitespace ban and won 33% of the length

Standard BPE can never build a token that crosses a pre-tokenization boundary. That restriction turned out to be unjustified.

- A curriculum: subwords first, then superwords bridging whitespace
- At a fixed 200k vocabulary: **up to -33% tokens** on the same text
- 8B model at equal train compute: **+4.0 pp** over 30 tasks, **+8.2 pp** on MMLU
- **-27%** inference compute as a side effect
- The price: training such a vocabulary is orders of magnitude slower

BPE: By the way, I am a fan of the Milky Way.

SuperBPE: By the way, I am a fan of the Milky Way.



Liu et al., COLM 2025, Fig. 1

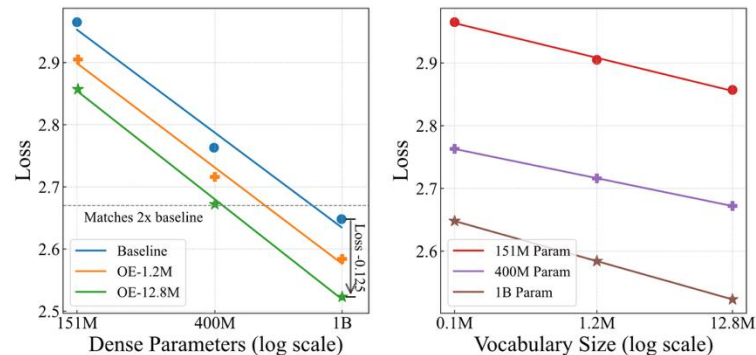
Input and output vocabularies can be decoupled — and the input is worth inflating

OVER-ENCODING

- A token's input = the sum of its 1-, 2- and 3-gram embeddings
- Input vocabulary $\times 128 \rightarrow$ a 400M model matches a 1B baseline's loss
- A log-linear relation between input vocabulary size and loss
- Almost no extra compute — the tables are used sparsely
- A large output vocabulary, by contrast, hurts small models

ALREADY IN PRODUCTION

- LongCat-Flash-Lite (Jan 2026) keeps most parameters in n-gram embeddings at $\sim 3B$ active
- Gemma 4 E2B/E4B: a related idea — capacity in tables, not in the stack



Huang et al., ICML 2025, Fig. 1



A tokenizer can be transplanted — the hard part is embedding initialization

A model is welded to its vocabulary only until you can guess the embeddings for the new tokens.

- **Heuristics** — WECHSEL, FOCUS: a new token as a mix of old subwords
- **ZeTT** — a hypernetwork takes a tokenizer and emits the embedding matrix
- **OMP** — training-free sparse reconstruction; shipped in mergekit-tokensurgeon
- **ALM** — cross-tokenizer distillation via approximate likelihood matching
- The worst thing to transfer is **digit tokenization** — it breaks maths

Token	Sparse Linear Decomposition
读者	$\approx$ 'readers' $\times$ 0.147 + 'reader' $\times$ 0.129 + '作者' $\times$ 0.126 + '음' $\times$ 0.139 + '学生' $\times$ 0.111 + 'retail' $\times$ 0.112 + 'visually' $\times$ 0.107 + '讀' $\times$ 0.149
불구하고	$\approx$ '그러나' $\times$ 0.142 + '인데' $\times$ 0.122 + '통해' $\times$ 0.101 + 'らず' $\times$ 0.094 + 'Despite' $\times$ 0.090 + 'のに' $\times$ 0.106 + 'является' $\times$ 0.090 + '-D' $\times$ -0.073
它是	$\approx$ '它' $\times$ 0.245 + '是' $\times$ 0.126 + '这是' $\times$ 0.154 + '{ }' $\times$ 0.111 + 'Exist' $\times$ 0.106 + 'metro' $\times$ 0.108 + 'cht' $\times$ -0.095 + '地说' $\times$ 0.091
'várias'	$\approx$ 'varias' $\times$ 0.208 + 'diversas' $\times$ 0.151 + 'muit' $\times$ 0.125 + 'varios' $\times$ 0.161 + 'uma' $\times$ 0.089 + 'various' $\times$ 0.090 + 'ários' $\times$ 0.097 + 'Brief' $\times$ -0.077
消者	$\approx$ 'consumers' $\times$ 0.122 + '市' $\times$ 0.099 + '고객' $\times$ 0.113 + '企' $\times$ 0.101 + 'Consumers' $\times$ 0.112 + '篇' $\times$ -0.072 + 'visitors' $\times$ 0.080 + 'portions' $\times$ -0.078
营利	$\approx$ '-profit' $\times$ 0.141 + '利' $\times$ 0.124 + 'ylim' $\times$ 0.127 + 'lucrative' $\times$ 0.130 + 'Windows' $\times$ 0.097 + 'purpos' $\times$ 0.147 + '营' $\times$ 0.095 + '-selling' $\times$ 0.106
专家	$\approx$ 'experts' $\times$ 0.166 + 'expert' $\times$ 0.163 + '教师' $\times$ 0.119 + '主任' $\times$ 0.118 + '政策' $\times$ 0.095 + 'Scientists' $\times$ 0.125 + '咨询' $\times$ 0.105 + '资产' $\times$ 0.098
必要があります	$\approx$ 'できます' $\times$ 0.197 + '있습니다' $\times$ 0.164 + 'необходимо' $\times$ 0.114 + 'harus' $\times$ 0.092 + 'べき' $\times$ 0.126 + 'However' $\times$ 0.086 + 'dispositivo' $\times$ 0.115 + 'muss' $\times$ 0.092
氟	$\approx$ 'fluoride' $\times$ 0.123 + '弗' $\times$ 0.150 + 'lithium' $\times$ 0.141 + '氧' $\times$ 0.112 + 'aa' $\times$ 0.139 + 'ann' $\times$ -0.103 + 'fluor' $\times$ 0.134 + 'dynam' $\times$ -0.098
'geführt'	$\approx$ 'gemacht' $\times$ 0.161 + 'führt' $\times$ 0.202 + '한다' $\times$ 0.097 + 'geben' $\times$ 0.088 + 'であり' $\times$ 0.076 + 'chnitt' $\times$ 0.089 + 'iliated' $\times$ 0.077 + 'Sch' $\times$ -0.061

Goddard et al., 2025, arXiv:2506.06607, Fig. 1

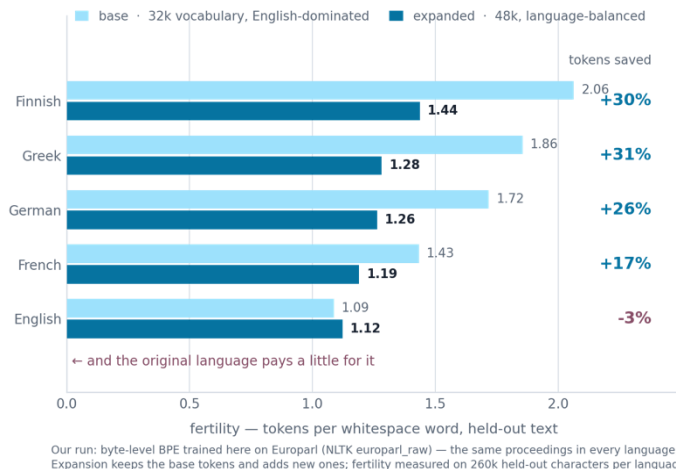
Grow the vocabulary for a language, trim it for a domain

EXPANSION

- Add frequent Russian tokens, initialise, continue training on 0.1–1B tokens
- Vocabulary reallocation cuts fertility for underserved languages by 15–30%
- The cost: a distribution shift and a risk of regressing on the original language

TRIMMING

- Drop tokens absent from the domain — embedding and LM head both shrink
- For a 262k vocabulary on a 1–3B model this frees a visible share of parameters
- Pays off far more on small models — on large ones embeddings are pocket change

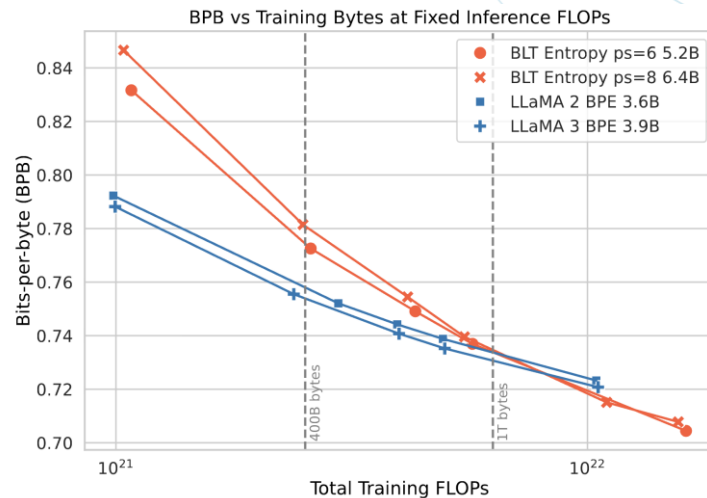


Industry examples: Teuken-7B trained a balanced 24-language vocabulary for fertility; Sarvam 30B/105B did the same for Indic languages



Or drop the tokenizer and let the model cut bytes itself

- **BLT** — entropy-based patches: predictable spans get longer, hard ones shorter
- **H-Net** — dynamic chunking learned end-to-end, no external entropy model
- **Bolmo** — «byteifying» an already trained subword model without losing quality
- Upside: cross-language fertility inequality disappears as a category
- Downside: sequences get longer, and attention is quadratic in that



Pagnoni et al., ACL 2025, Fig. 1

No frontier model released by August 2026 ships without a tokenizer. A live and elegant direction that still loses on engineering — a good reading-group candidate.

03

• Embeddings: the entry to the residual stream

A $V \times d$ matrix usually covered in one line — and it can be half the model



An embedding is a lookup, not a matrix multiply

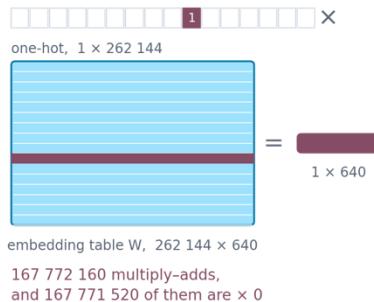
THE IDENTITY

- Formally: a one-hot vector times a $V \times d$ matrix
- Actually: take row i — the same number, thousands of times cheaper
- Hence «parameters are almost free in compute»: they are read, not multiplied
- PLE and n-gram embeddings exploit exactly this property

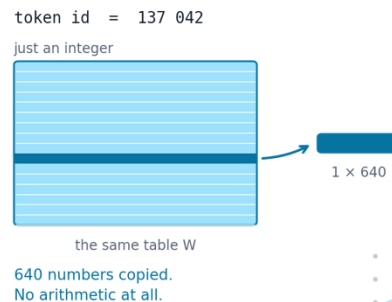
WHY IT MATTERS LATER

- The same argument explains active vs total parameters in MoE — lecture 2
- And why capacity in tables is cheaper than capacity in the stack

what the textbook writes



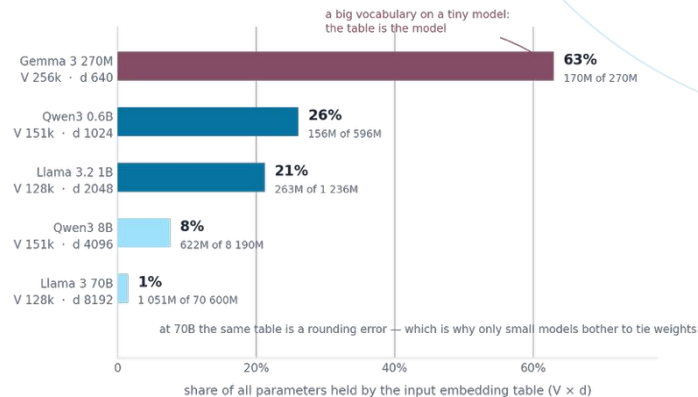
what the hardware does



In a small model, embeddings are more than half the parameters

Gemma 3 270M: vocabulary 262 144, $d = 640$. The input table alone is about 168M of 270M.

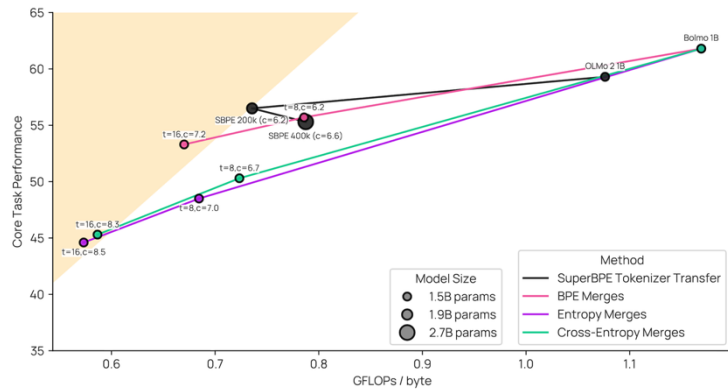
- Contribution to the budget: $V \cdot d$ on input, another $V \cdot d$ on the LM head
- For a 270M model with a large vocabulary this is the dominant line item
- For 70B it is a couple of percent — no need to think about it
- Hence «effective parameters» are counted separately for small models



Our computation: $V \times d$ from published configs, against total parameter count. Gemma 3 270M figure is Google's own (170M embedding of 270M). Models that tie input and output weights reuse this one table as the LM head; models that do not pay for it a second time.

The LM head costs more than it looks: a softmax over 262k every step

- Every generated token: a $d \times V$ multiply and a softmax over V
- On a small model with a big vocabulary this rivals the entire block stack
- For byte-level models this is exactly where the vocabulary gain breaks
- Hence chunked cross-entropy, cut cross-entropy, never materialising the logits
- And hence Over-Decoding: a large output vocabulary only helps large models



Bolmo, 2025, Fig. 3



The embedding is the first write into the residual stream, not a preprocessing step



the whole stack only reads from it and writes back · we return to this framing in
lecture 14



04

• Positions: the most underrated detail

Usually given one slide. It decides whether your long context works at all



Without positions, attention sees a bag of tokens

Self-attention is permutation-equivariant: permute the input and the output permutes identically — the content does not change.

- «dog bit man» and «man bit dog» are the same bag
- Absolute learned and sinusoidal: added to the embedding, fine up to train length
- Beyond the trained length they do not extrapolate — quality falls off a cliff
- ALiBi: a linear distance penalty in the logits; strong extrapolation, displaced by RoPE + extension

Swap the words and the attention output just swaps with them

no positional information · “dog bit man”



no positional information · “man bit dog”



the vector for “dog” is the same in both sentences · $\max |\text{difference}| = 2.2 \times 10^{-16}$, i.e. floating-point noise

with RoPE · “dog bit man”



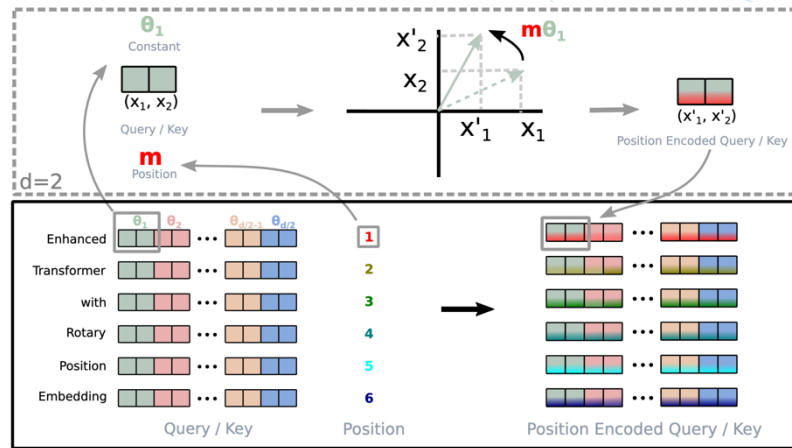
with RoPE · same word, other sentence: $\max |\text{difference}| = 1.06$ — now the representation actually moves

RoPE: position is a rotation, not an addition

- Q and K split into 2D subspaces; each rotates by an angle $\propto$ position
- In the dot product the absolute angles cancel — the **difference** remains
- Relativity emerges by itself, with no distance table
- Key point: this happens inside attention, it is not a property of the embedding
- V is not rotated — only Q and K, because position is needed in the scores

$$f_{\{q,k\}}(\mathbf{x}_m, m) = \mathbf{R}_{\Theta, m}^d \mathbf{W}_{\{q,k\}} \mathbf{x}_m$$

$$\mathbf{R}_{\Theta, m}^d = \begin{pmatrix} \cos m\theta_1 & -\sin m\theta_1 & 0 & 0 & \cdots & 0 & 0 \\ \sin m\theta_1 & \cos m\theta_1 & 0 & 0 & \cdots & 0 & 0 \\ 0 & 0 & \cos m\theta_2 & -\sin m\theta_2 & \cdots & 0 & 0 \\ 0 & 0 & \sin m\theta_2 & \cos m\theta_2 & \cdots & 0 & 0 \\ \vdots & \vdots & \vdots & \vdots & \ddots & \vdots & \vdots \\ 0 & 0 & 0 & 0 & \cdots & \cos m\theta_{d/2} & -\sin m\theta_{d/2} \\ 0 & 0 & 0 & 0 & \cdots & \sin m\theta_{d/2} & \cos m\theta_{d/2} \end{pmatrix}$$

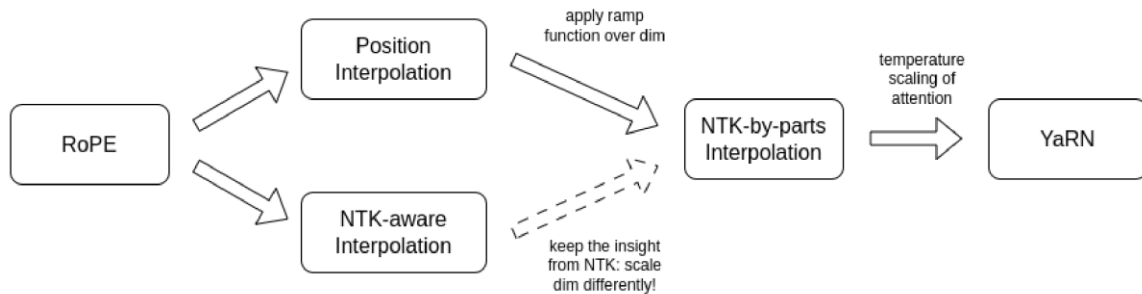


Su et al., 2021, RoFormer, Fig. 1

The θ base is the only knob everyone pulls to extend context

θ sets the wavelengths of the rotation. Past the positions the model has seen, the angles enter an unfamiliar regime and quality collapses.

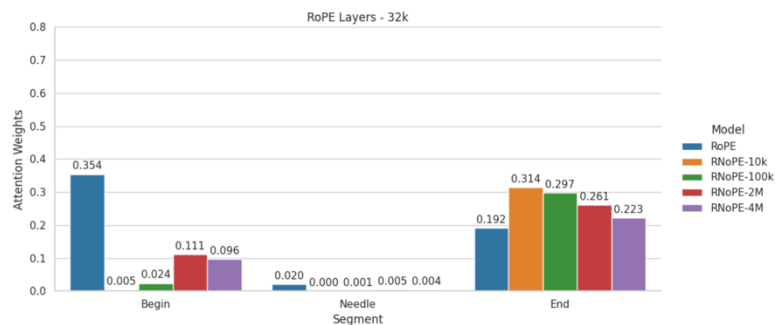
- **PI** — squeeze the position indices back into the trained range
- **NTK-aware** — change the base unevenly across frequencies
- **YaRN** — the same plus an attention temperature; the de-facto standard



RoPE creates a recency bias, and it hurts needle retrieval

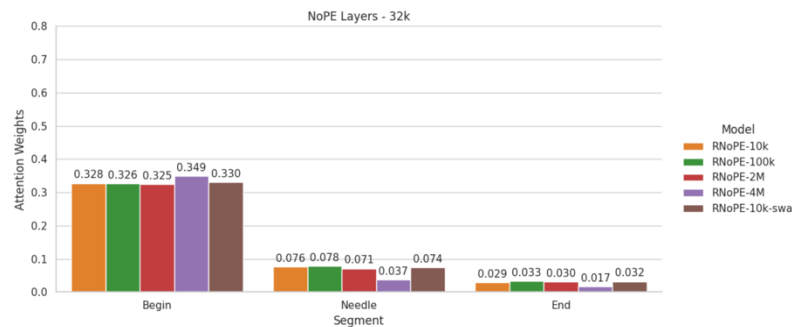
THE MECHANISM

- Large rotations pull distant Q and K apart — scores decay with distance
- As an inductive bias for language, that is mostly a plus
- As behaviour for retrieval at 100k+, it is a minus



NOPE: REMOVE POSITIONS ENTIRELY

- The causal mask leaks positional information — the model reconstructs it
- With no recency bias, a token 3 away and 80 000 away are equally reachable
- The price: weak extrapolation, and worse where exact order matters

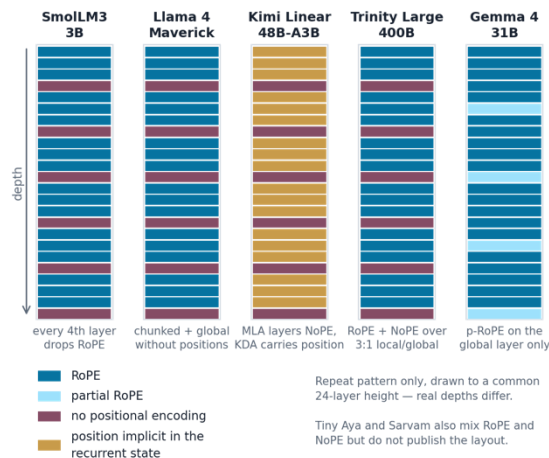


So the frontier stopped choosing and started mixing

Model	Date	What it does with positions
SmolLM3 (3B)	06.2025	every fourth layer drops RoPE
Llama 4 Maverick	04.2025	chunked layers with RoPE + global layers without
Kimi Linear (48B-A3B)	10.2025	MLA layers run NoPE; position lives in KDA
Tiny Aya (3.35B)	02.2026	RoPE and NoPE mixed, parallel block
Trinity Large (400B)	01.2026	RoPE + NoPE over a 3:1 sliding/global stack
Sarvam (105B)	03.2026	MLA with NoPE + RoPE and KV LayerNorm
Gemma 4 (31B)	04.2026	p-RoPE on global layers only

One repeat unit per model

locals keep position, globals let it go



Position leaks into the model through three paths besides RoPE

THE THREE PATHS

- **The causal mask** — the shape of the triangle carries order by itself
- **Sliding window** — a 128 or 512-token window is a hard local prior
- **Convolutions in the block** — a kernel-4 ShortConv in KDA sets local order

WHY IT MATTERS

- This is why decoupled RoPE inside MLA works: a few channels hold position, the rest compresses into a latent
- And why NoPE layers are usable at all — they are not position-blind, just position-unbiased



05

Attention and its two costs

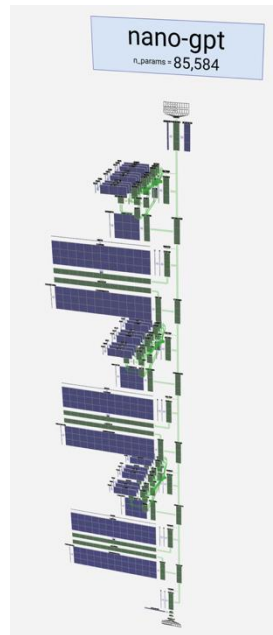
Why the quadratic term is not an abstract complaint but a line in the GPU bill



ATTENTION

Q, K and V are three questions asked of one residual stream

- **Q** — what I look for • **K** — the key I answer to • **V** — what I pass on
- $\text{Score} = \mathbf{Q} \cdot \mathbf{K}^T / \sqrt{d_k}$; the divisor keeps the variance from saturating softmax
- Heads split d between them: different heads catch different relations
- Induction heads, syntactic heads, positional heads — that is lecture 14
- Per MHA layer: $4d^2$ parameters in W_Q, W_K, W_V, W_O



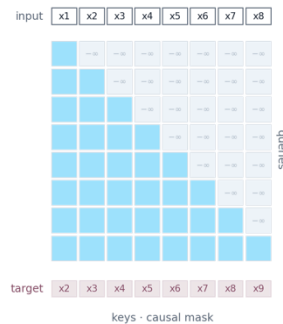
LLM Visualization, Bycroft · bycroft.net/llm

ATTENTION

The causal mask is what makes training parallel, inference one by one

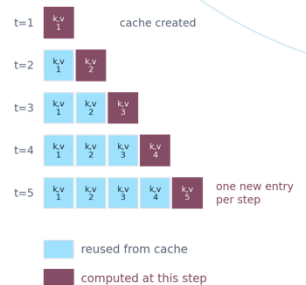
- Lower-triangular: position t sees only $1 \dots t$
- $-\infty$ is added **before** softmax — after it, the zeros would not stay zero
- One forward computes the loss over all positions, with the target shifted by one
- Encoder-decoder died in LLMs for this reason: decoder-only gets signal per token
- Side effect: the mask leaks positional information — hence NoPE

Training — one forward, all positions



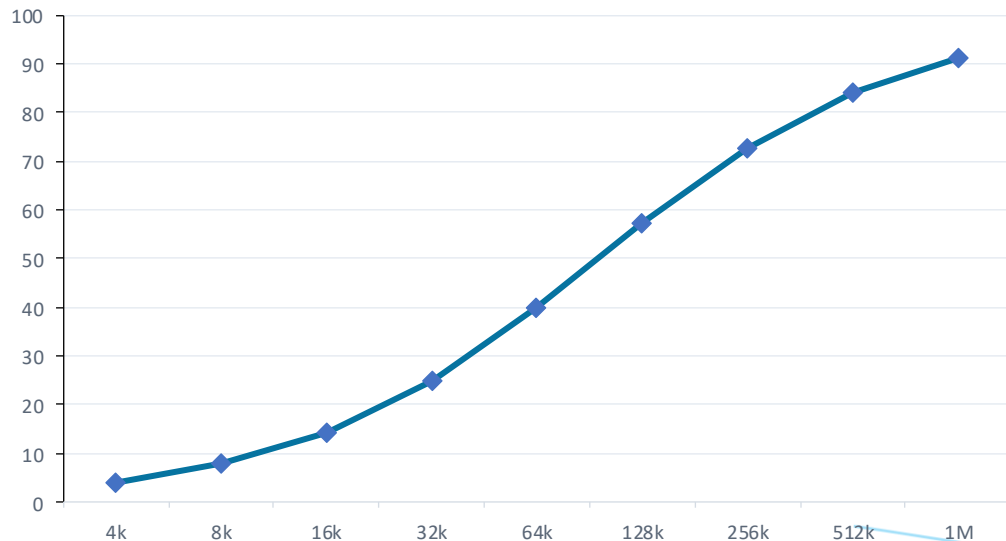
the loss is taken at all 8 positions
in the same forward pass

Inference — n steps, prefix reused



the prefix is never recomputed — that is
why causality is what makes a cache possible

At 128k, attention eats more than half the compute



POINTS WORTH REMEMBERING

- 4k → 4%. Nobody worried about attention for a decade
- 128k → 57%. Agentic lengths flipped the balance
- 1M → 91%. The block stack becomes a rounding error

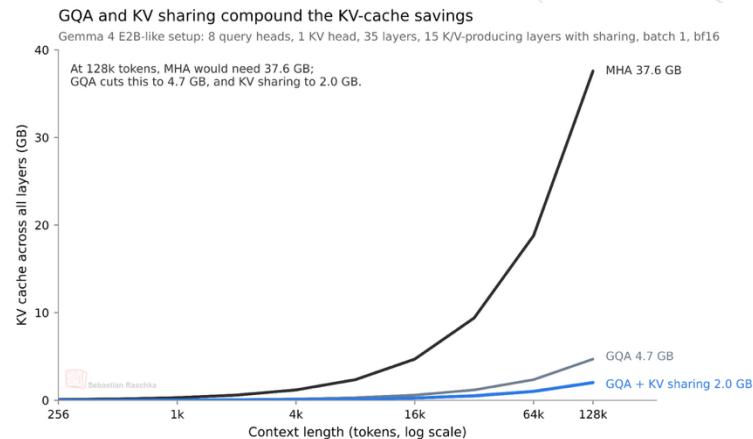
Computation for a dense model with $d = 8192$: $\text{share} = n / (12d + n)$. SwiGLU $d_{\text{ff}} = 8/3 \cdot d$, averaged over the causal mask, GQA savings on projections not counted.



The KV cache is linear in length — and it is what kills the batch

Size = $2 \cdot n_{\text{layers}} \cdot n_{\text{kv_heads}} \cdot d_{\text{head}} \cdot \text{seq} \cdot \text{batch} \cdot \text{dtype}$

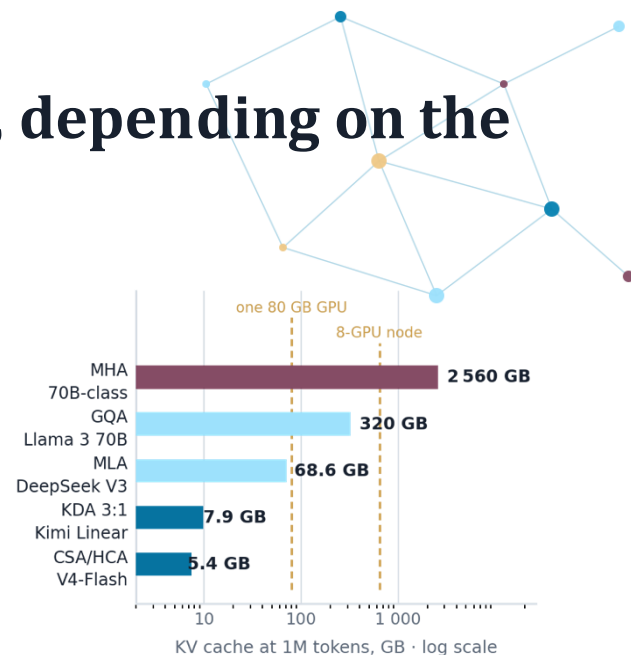
- Llama 3 70B: 80 layers, 8 KV heads, d_{head} 128 → **320 KiB per token**
- At 128k that is **40 GB** for one sequence, before any batching
- Batch 8 is already 320 GB — four H100s for the cache alone
- Weights load once; the cache grows per request — so it caps throughput



Raschka, 2026, Fig. 5 — savings at 128k

The same context costs 320 GB or 0.7 GB, depending on the year

Attention	Model	4k	128k	1M
MHA	70B-class	10 GB	320 GB	2 560 GB
GQA	Llama 3 70B	1.3 GB	40 GB	320 GB
MLA	DeepSeek V3	0.3 GB	8.6 GB	68.6 GB
KDA 3:1	Kimi Linear	0.03 GB	1.0 GB	7.9 GB
CSA/HCA	V4-Flash	0.02 GB	0.7 GB	5.4 GB

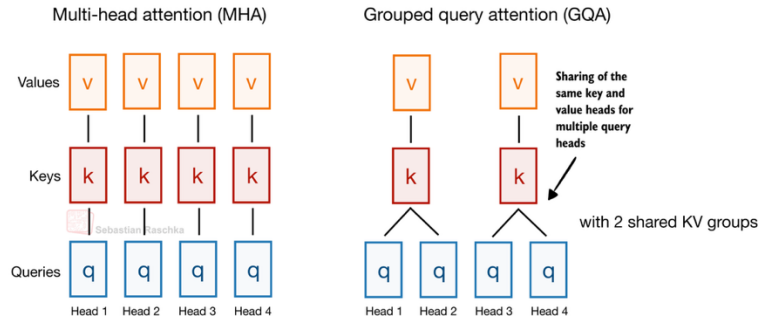


Our computation from KiB-per-token figures in the LLM Architecture Gallery, bf16, batch = 1. At 4k the generations are indistinguishable; at 1M only the last two are shippable.

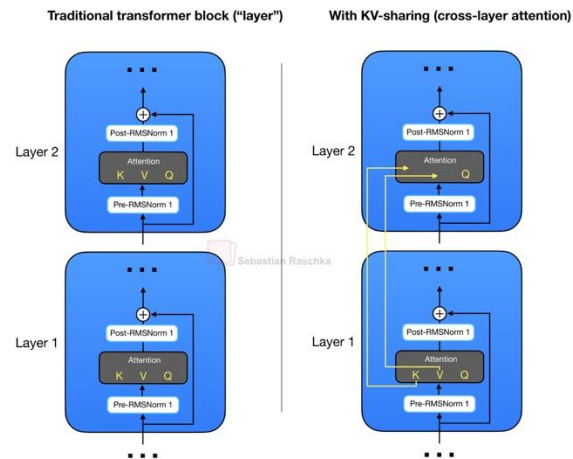


GQA: a group of query heads shares one K/V pair

- MQA is one KV head for all; MHA is one per query head
- GQA interpolates: g groups, cache smaller by h/g
- Llama 3 70B: 64 query heads over 8 KV heads $\rightarrow$ 8 $\times$ less cache than MHA
- The 2023–2025 default: Llama, Qwen3, Gemma, GLM-4.5, gpt-oss
- Gemma 4 goes further: later layers **reuse** the KV of earlier ones — -2.7 GB at 128k on E2B



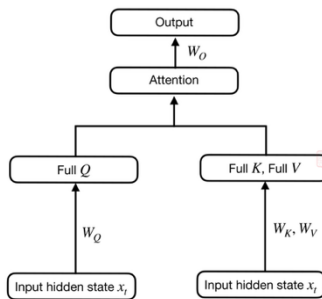
Raschka, 2026, Fig. 3 (GQA) and Fig. 4 (KV sharing)



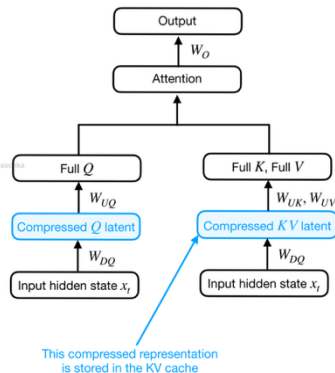
MLA caches a latent instead of K and V

- K and V are projected into a low-rank latent, and the latent is what is cached
- Position is split into a separate RoPE branch — otherwise the latent would not compress
- DeepSeek V3 and Kimi K2: **68.6 KiB per token** at 61 layers — less than a dense 8B on GQA
- Spread widely: Mistral Large 3, GLM-5, Mistral Small 4, Sarvam 105B
- ZAYA1-8B goes further: attention computed inside the compressed space (CCA), saving FLOPs too

Regular Multi-head Attention (MHA)
(only one attention head shown)



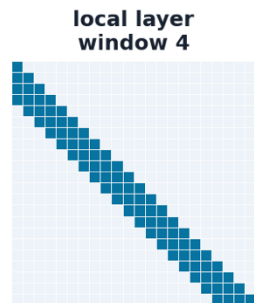
Multi-head **Latent** Attention (MLA)
(only one attention head shown)



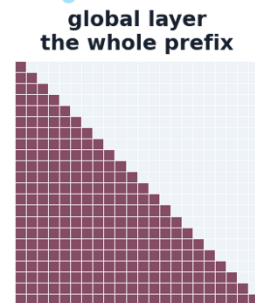
Raschka, 2026, Fig. 12

Sliding window plus global is the cheapest hybrid that works

Model	Layer mix	Window	KV/token
Gemma 3 (27B)	52 local + 10 global, 5:1	1024	496 KiB
gpt-oss (120B)	18 local + 18 global, 1:1	128	72 KiB
OLMo 3 (32B)	48 local + 16 global, 3:1	4096	256 KiB
MiMo-V2-Flash (309B)	40 local + 8 global, 5:1	128	144 KiB
Laguna XS.2 (33B)	30 local + 10 global, 3:1	512	160 KiB



caches 4 entries per token



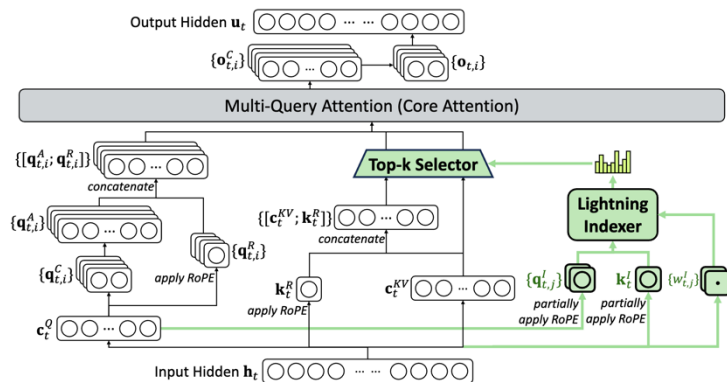
caches every token so far

Model configs, LLM Architecture Gallery, 23 Aug 2026. Laguna also budgets query heads by layer: 8 per local layer, 6 per global one — capacity where it is cheap.



DSA: do not compress the cache — select the top-k from it

- A lightning indexer cheaply scores how relevant each past token is to the query
- Attention runs over the selected entries only — compute stops being quadratic
- The cache stays full: the saving is in FLOPs, not in memory
- DeepSeek V3.2 (Dec 2025) shipped it first; GLM-5 (Feb 2026) adopted it
- Sparsity here is **learned and dynamic**, unlike a fixed window

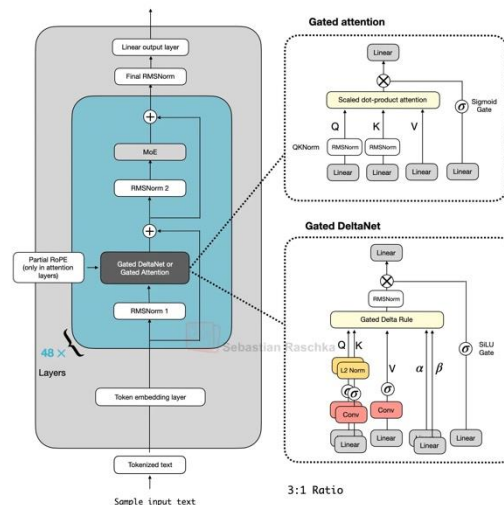


DeepSeek-V3.2, 2025, Fig. 2

Three quarters of Qwen and Kimi layers no longer run softmax attention

- The same recipe everywhere: **3 linear layers per 1 full one**
- Qwen3-Next / 3.5 / 3.6 / 3.8 — Gated DeltaNet plus Gated Attention
- Kimi Linear — Kimi Delta Attention plus MLA, gating per channel, not per head
- Ling 2.5 (1T) pushes to 7:1 with Lightning Attention
- Nemotron 3 is Mamba-2 with only 6–8 attention layers out of 50+
- The price: a linear layer keeps a **fixed-size** state — it cannot revisit one old token

Qwen3 Next 80B-A3B



3:1 Ratio

Layer 1 : Linear attention + MoE
 Layer 2 : Linear attention + MoE
 Layer 3 : Linear attention + MoE
 Layer 4 : Full attention + MoE
 Layer 5 : Linear attention + MoE
 Layer 6 : Linear attention + MoE
 Layer 7 : Linear attention + MoE
 Layer 8 : Full attention + MoE

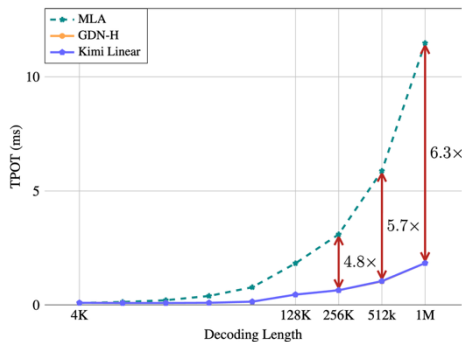
Kimi Linear: -75% KV cache and 6× decode throughput at 1M

WHAT WAS MEASURED

- 48B total / 3B active, 20 KDA layers to 7 MLA layers
- **7.9 KiB per token** against 68.6 for MLA models
- Up to **6×** decode throughput at 1M context
- The first fair match where a linear hybrid beats full attention on short, long and RL regimes at an identical recipe

HOW MUCH TO TRUST IT

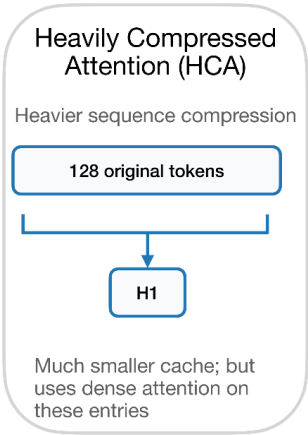
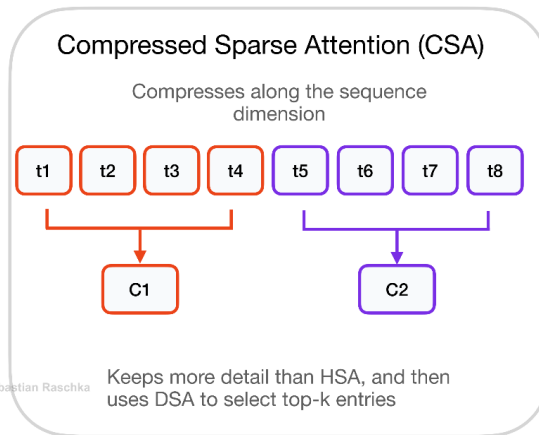
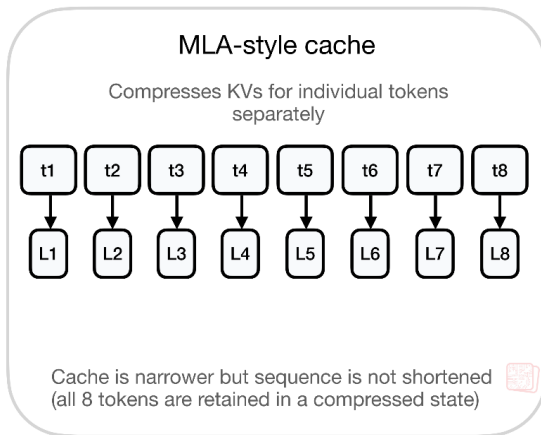
- The claim is the authors' own, at their recipe and their data
- No independent reproduction at comparable scale yet
- Kimi K3 (Jul 2026, 2.8T) is built on KDA — the idea reached the flagship in 9 months



DeepSeek V4 compresses the sequence, not the token

MLA kept one entry per token. CSA and HCA shorten the number of entries: a group of tokens becomes one.

- **CSA** — every $m = 4$ tokens into one entry, then top-k = 1024 in DSA style
- **HCA** — $m' = 128$ tokens per entry, but attention over them is dense
- Both branches keep a 128-token sliding window over recent tokens
- Layers alternate: HCA as a long index, CSA as the precise lookup
- Plus mHC — four parallel residual streams instead of one

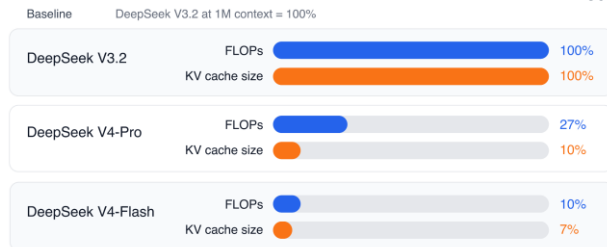


At a million tokens V4 costs a quarter of what V3.2 costs

27%

FLOPs per token

V4-Pro relative to V3.2 at 1M context.



Values are relative to DeepSeek V3.2 in the DeepSeek V4 paper, not absolute FLOP or cache values.

Raschka, 2026, Fig. 23 · DeepSeek-V4 paper

10%

KV cache size

Same comparison. V4-Flash: 10% FLOPs and 7% cache.

5.4

KiB per token

V4-Flash. GPT-2 XL needed 300 KiB.

DeepSeek-AI, 2026, arXiv:2606.19348. The paper contains no CSA/HCA ablation — the numbers belong to the full recipe, including data, Muon and mHC.



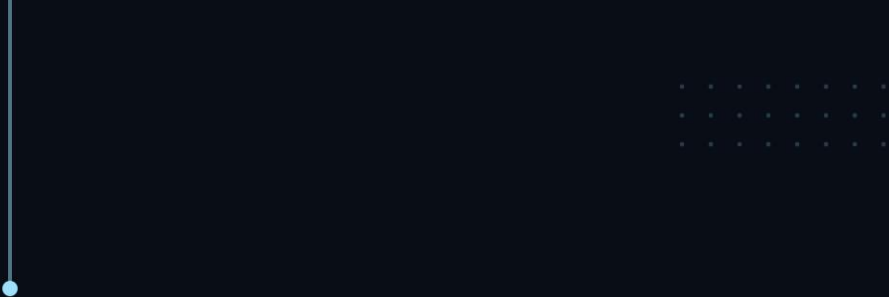
AUGUST 2026

There is no single winner — and that is an open question, not an oversight



Model	Date	Attention	KV/token
DeepSeek V4-Pro (1.6T)	24.04.26	CSA / HCA + mHC, 1M context	7.7 KiB
GLM-5.1 (744B)	07.04.26	MLA + DeepSeek Sparse Attention	87.8 KiB
Qwen3.6 (35B-A3B)	15.04.26	3:1 Gated DeltaNet / Gated Attention	20 KiB
Gemma 4 (31B)	02.04.26	5:1 SWA/global, p-RoPE, shared K/V	840 KiB
MiniMax M2.7 (230B)	18.03.26	plain GQA + QK-Norm — hybrids refused on purpose	248 KiB
Kimi K3 (2.8T)	07.2026	KDA + attention residuals	—
Qwen3.8-27B	14.08.26	3:1 GDN / Gated Attention, 262k → 1M via YaRN	—





Recent architecture work is about cost, not about quality

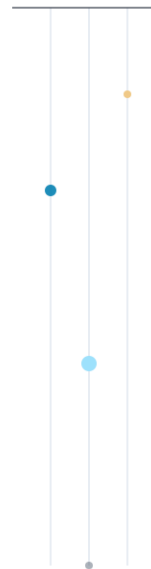


quality is pulled by data and the training recipe · architecture decides what it costs



Five things to remember

- 1 A tokenizer is part of the model, not preprocessing**
It sets length, cost and effective context. Russian adds ~70% to the bill.
- 2 Perplexity compares only within one tokenizer**
Otherwise use bits-per-byte.
- 3 Position lives inside attention, not in the embedding**
And half of the 2026 models leave some layers without it entirely.
- 4 Attention has two separate costs**
Compute is quadratic, cache is linear. Methods fix them separately.
- 5 You can compress the token or the sequence**
MLA is the first, CSA/HCA the second. Different trade-offs, not versions of one.



What to read and what to play with



Raschka

Recent Developments in LLM Architectures · LLM Architecture Gallery
05.2026 · sebastianraschka.com/llm-architecture-gallery

DeepSeek-AI

DeepSeek-V4: million-token context, CSA/HCA, mHC
2026 · [arXiv:2606.19348](https://arxiv.org/abs/2606.19348)

Kimi Team

Kimi Linear: an expressive, efficient attention architecture
2025 · [arXiv:2510.26692](https://arxiv.org/abs/2510.26692)

Liu et al.

SuperBPE: Space Travel for Language Models
COLM 2025 · [arXiv:2503.13423](https://arxiv.org/abs/2503.13423)

Petrov et al.

Language Model Tokenizers Introduce Unfairness Between Languages
NeurIPS 2023 · [arXiv:2305.15425](https://arxiv.org/abs/2305.15425)

Minixhofer et al.

Zero-Shot Tokenizer Transfer · Bolmo
[arXiv:2405.07883](https://arxiv.org/abs/2405.07883) · [arXiv:2512.15586](https://arxiv.org/abs/2512.15586)

Su et al. · Peng et al.

RoFormer (RoPE) · YaRN
[arXiv:2104.09864](https://arxiv.org/abs/2104.09864) · [arXiv:2309.00071](https://arxiv.org/abs/2309.00071)

Interactive

bbycroft.net/llm · Transformer Explainer · Tiktokenizer · minbpe
[links in the course README](#)



Lecture 2: the rest of the block

FFN and SwiGLU, MoE, normalisation, hyper-connections, the full forward pass and the parameter budget

seminar

today at 15:30 — a transformer from scratch

materials

course repository, slides go up today

reading

sign-up form in the chat

• • • • •
• • • • •
• • • • •

